

I. FILE OPERATIONS:	3
1. Create and navigate to a lab directory.....	3
Figure 1: Make Directory.....	3
2. Create & edit a file (nano).....	3
3. View file content.....	3
Figure 2: create and edit, view content.....	3
4. Copy & Rename.....	3
Figure 3: copy & move from backup storage.....	3
5. Remove file & Directories:.....	4
6. Check permissions:.....	4
Figure 4: Remove file & folder.....	4
II. APT PACKAGE MANAGEMENT:	4
1. Update the package list:.....	4
Figure 5: Update the package list.....	4
2. Install the C Compiler (GCC) with build-essential package.....	5
Figure 6: Install compiler.....	5
3. Verify installation.....	5
4. Search for other package (apt search).....	6
Figure 8: Search package.....	6
III. C COMPILATION:	7
A. Step 1: Write the code.....	7
1. Create source file & write the code.....	7
Figure 9: Create source file.....	7
2. Write the code:.....	7
Figure 10: C Program.....	7
B. Step 2: Compile & Run.....	8
3. Compile the code.....	8
4. Check the executable was created.....	8
Figure 11: Compile & Executable.....	8
5. Run the compiled program.....	8
Figure 12: Run program.....	8
6. Compile with warning enabled.....	8
Figure 13: Compile warning enabled.....	8
C. Step 3: Modify and Recompile.....	9
1. Edit the file to change the numbers.....	9
Figure 14: Edit program.....	9
2. Recompile.....	9
3. Run again to see changes.....	9
Figure 15: Recompile & Run to see change.....	9

IV. CLEANUP AND VERIFICATION:	10
1. Go back to home directory	10
2. List all files in the lab directory	10
Figure 16: Go back & List all files	10
3. View the content of your C file one last time	10
Figure 17: View C program	10
4. Remove the entire lab directory	11
Figure 18: Remove all lab directory	11
SUMMARY:	11

I. FILE OPERATIONS:

1. Create and navigate to a lab directory

```
ubuntu@ubuntu:~$ mkdir linux_lab
ubuntu@ubuntu:~$ cd linux_lab
ubuntu@ubuntu:~/linux_lab$ pwd
/home/ubuntu/linux_lab
```

Figure 1: Make Directory

2. Create & edit a file (nano)

- Type txt. then (save Ctrl+X and exit for press enter)

3. View file content

```
ubuntu@ubuntu:~/linux_lab$ nano greeting.txt
ubuntu@ubuntu:~/linux_lab$
ubuntu@ubuntu:~/linux_lab$ nano greeting.txt
ubuntu@ubuntu:~/linux_lab$ cat greeting.txt
"Hello linux"
```

Figure 2: create and edit, view content

4. Copy & Rename

```
ubuntu@ubuntu:~/linux_lab$ cp greeting.txt backup.txt
ubuntu@ubuntu:~/linux_lab$ ls
backup.txt  greeting.txt
ubuntu@ubuntu:~/linux_lab$ mv backup.txt old_greeting.txt
mv: target 'greeting.txt': Not a directory
ubuntu@ubuntu:~/linux_lab$ mv backup.txt old_greeting.txt
ubuntu@ubuntu:~/linux_lab$ ls
greeting.txt  old_greeting.txt
```

Figure 3: copy & move from backup storage

5. Remove file & Directories:
6. Check permissions:

```
ubuntu@ubuntu:~/linux_lab$ rm old_greeting.txt
ubuntu@ubuntu:~/linux_lab$ ls
greeting.txt
ubuntu@ubuntu:~/linux_lab$ mkdir temp_dir
ubuntu@ubuntu:~/linux_lab$ ls
greeting.txt  temp_dir
ubuntu@ubuntu:~/linux_lab$ ls -l greeting.txt
-rw-rw-r-- 1 ubuntu ubuntu 14 Sep  8 12:18 greeting.txt
```

Figure 4: Remove file & folder

II. APT PACKAGE MANAGEMENT:

1. Update the package list:

```
ubuntu@ubuntu:~/linux_lab$ sudo apt update
Ign:1 file:/cdrom/resolute InRelease
Get:2 file:/cdrom/resolute Release [418 B]

f Metadata [120 B]
Fetched 42.8 MB in 11s (4016 kB/s)
101 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 5: Update the package list

2. Install the C Compiler (GCC) with build-essential package

```
ubuntu@ubuntu:~/linux_lab$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  dpkg-dev                libalgorithm-diff-perl      libhwasan0
  fakeroot                libalgorithm-diff-xs-perl   libitm1
  g++                     libalgorithm-merge-perl     liblsan0
  g++-15                  libasan8                     libstdc++-15-dev
  g++-15-aarch64-linux-gnu libcc1-0                     libtsan2
  g++-aarch64-linux-gnu   libcrypt-dev                 libubsan1
  gcc                     libdpkg-perl                 lto-disabled-list
  gcc-15                  libfakeroot                  make
  gcc-15-aarch64-linux-gnu libfile-fcntllock-perl
  gcc-aarch64-linux-gnu   libgcc-15-dev

Suggested packages:
  debian-keyring          libtool                      git
  debian-tag2upload-keyring flex                          bzip2
  gcc-15-doc              bison                       libstdc++-15-doc
  gcc-multilib            gcc-doc                     make-doc
  autoconf                gcc-15-locales
  automake                gdb-aarch64-linux-gnu
```

Figure 6: Install compiler

3. Verify installation

```
ubuntu@ubuntu:~/linux_lab$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 7: Verify installation

4. Search for other package (apt search)

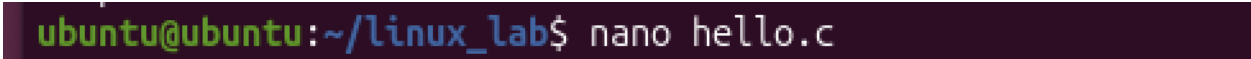
```
ubuntu@ubuntu:~/linux_lab$ apt search python3  
alembic/resolute 1.18.4-1 all  
  lightweight database migration tool for SQLAlchemy  
  
antimony/resolute 0.9.3-2.1build2 arm64  
  Computer-aided design CAD tool  
  
astro-frameworks/resolute 5.0 all  
  Astronomical data analysis frameworks  
  
astro-python3/resolute 5.0 all  
  Python 3 packages for astronomy  
  
bamkit/resolute 0.0.1+git20170413.ccd079d-3build1 all  
  tools for common BAM file manipulations
```

Figure 8: Search package

III. C COMPILATION:

A. Step 1: Write the code

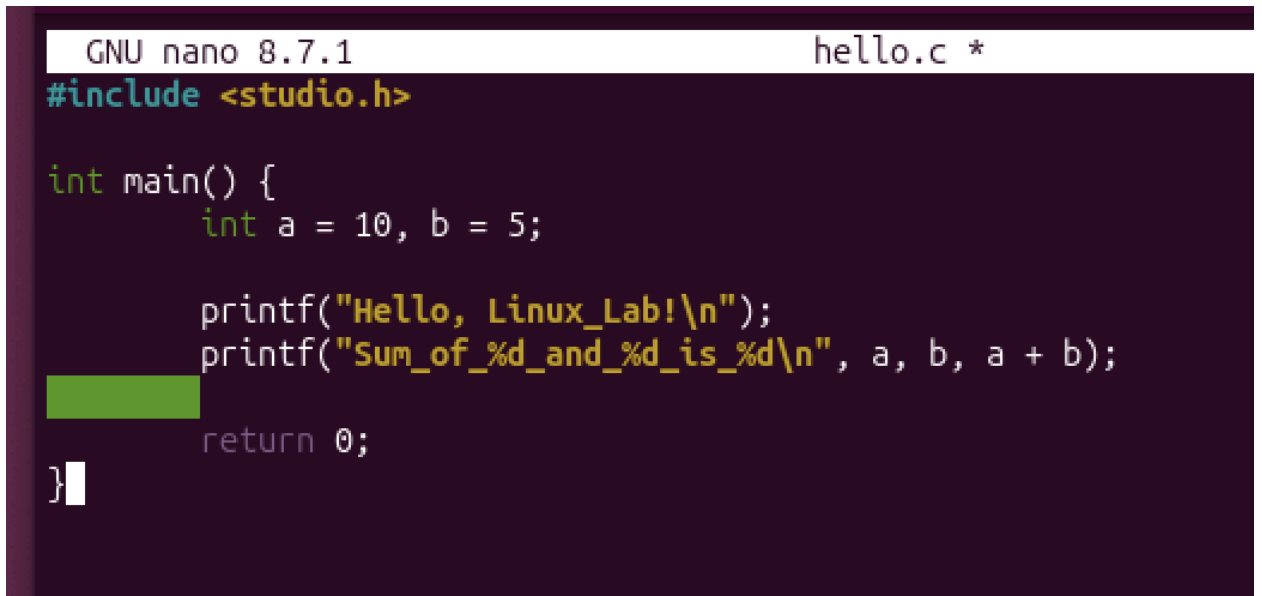
1. Create source file & write the code



```
ubuntu@ubuntu:~/linux_lab$ nano hello.c
```

Figure 9: Create source file

2. Write the code:



```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main() {
    int a = 10, b = 5;

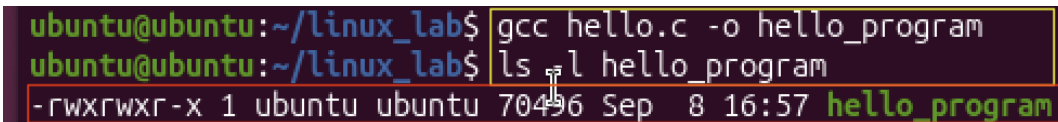
    printf("Hello, Linux_Lab!\n");
    printf("Sum_of_%d_and_%d_is_%d\n", a, b, a + b);
    return 0;
}
```

Figure 10: C Program

B. Step 2: Compile & Run

3. Compile the code

4. Check the executable was created

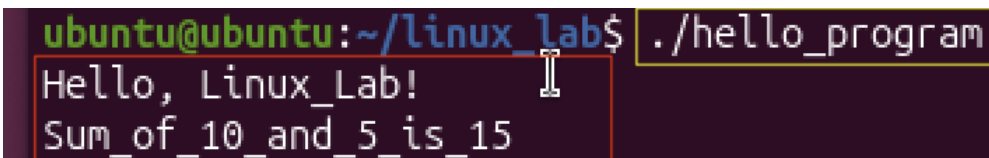


```
ubuntu@ubuntu:~/linux_lab$ gcc hello.c -o hello_program
ubuntu@ubuntu:~/linux_lab$ ls -l hello_program
-rwxrwxr-x 1 ubuntu ubuntu 70496 Sep  8 16:57 hello_program
```

A terminal window with a dark background. The first line shows the command 'gcc hello.c -o hello_program' being executed. The second line shows 'ls -l hello_program' being executed, followed by the output listing the file 'hello_program' with permissions '-rwxrwxr-x', owner 'ubuntu', group 'ubuntu', size '70496', and timestamp 'Sep 8 16:57'.

Figure 11: Compile & Executable

5. Run the compiled program

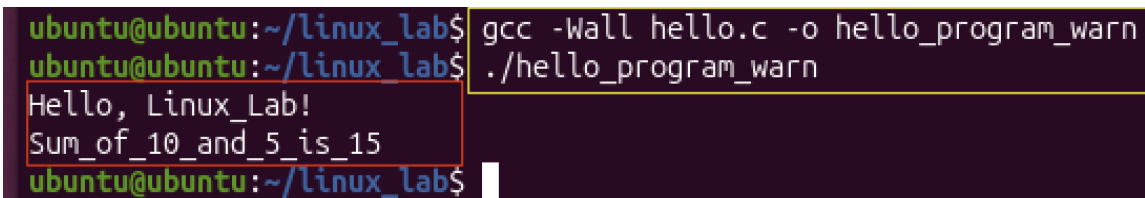


```
ubuntu@ubuntu:~/linux_lab$ ./hello_program
Hello, Linux_Lab!
Sum_of_10_and_5_is_15
```

A terminal window showing the execution of './hello_program'. The output consists of two lines: 'Hello, Linux_Lab!' and 'Sum_of_10_and_5_is_15'.

Figure 12: Run program

6. Compile with warning enabled



```
ubuntu@ubuntu:~/linux_lab$ gcc -Wall hello.c -o hello_program_warn
ubuntu@ubuntu:~/linux_lab$ ./hello_program_warn
Hello, Linux_Lab!
Sum_of_10_and_5_is_15
ubuntu@ubuntu:~/linux_lab$
```

A terminal window showing the compilation and execution of the program with warnings enabled. The first line shows 'gcc -Wall hello.c -o hello_program_warn'. The second line shows './hello_program_warn'. The output is the same as in Figure 12: 'Hello, Linux_Lab!' and 'Sum_of_10_and_5_is_15'. The prompt returns after the execution.

Figure 13: Compile warning enabled

C. Step 3: Modify and Recompile

1. Edit the file to change the numbers

- Change a=10 to a=20,
- Then, save & exit



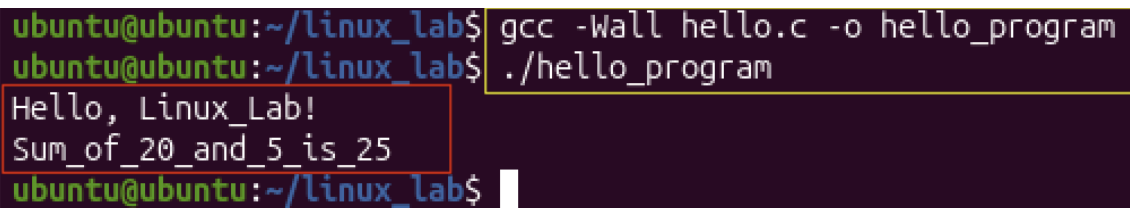
```
ubuntu@ubuntu:~/linux_lab$ nano hello.c
```

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/linux_lab\$' and the command 'nano hello.c' is entered.

Figure 14: Edit program

2. Recompile

3. Run again to see changes



```
ubuntu@ubuntu:~/linux_lab$ gcc -Wall hello.c -o hello_program
ubuntu@ubuntu:~/linux_lab$ ./hello_program
Hello, Linux_Lab!
Sum_of_20_and_5_is_25
ubuntu@ubuntu:~/linux_lab$
```

A terminal window with a dark purple background. The first two lines are the compilation and execution commands, highlighted with a yellow box. The next two lines are the program's output, highlighted with a red box. The final line is the prompt after the program has finished.

Figure 15: Recompile & Run to see change

IV. CLEANUP AND VERIFICATION:

1. Go back to home directory
2. List all files in the lab directory

```
ubuntu@ubuntu:~$ cd ~
ubuntu@ubuntu:~$ ls -lh ~/linux lab/
total 40K
-rw-rw-r-- 1 ubuntu ubuntu 14 Sep  8 12:18 greeting.txt
-rw-rw-r-- 1 ubuntu ubuntu 151 Sep  8 18:57 hello.c
-rwxrwxr-x 1 ubuntu ubuntu 69K Sep  8 18:59 hello_program
-rwxrwxr-x 1 ubuntu ubuntu 69K Sep  8 17:08 hello_program_warn
```

Figure 16: Go back & List all files

3. View the content of your C file one last time

```
ubuntu@ubuntu:~$ cat ~/linux_lab/hello.c
#include <stdio.h>

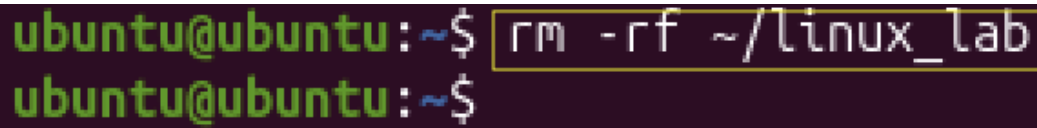
int main() {
    int a = 20, b = 5;

    printf("Hello, Linux_Lab!\n");
    printf("Sum_of_%d_and_%d_is_%d\n", a, b, a + b);

    return 0;
}
```

Figure 17: View C program

4. Remove the entire lab directory

A terminal window with a dark purple background. The prompt 'ubuntu@ubuntu:~\$' is shown in green. The command 'rm -rf ~/linux_lab' is entered in white and highlighted with a yellow rectangular box. The prompt 'ubuntu@ubuntu:~\$' is shown again on the next line.

```
ubuntu@ubuntu:~$ rm -rf ~/linux_lab
ubuntu@ubuntu:~$
```

Figure 18: Remove all lab directory

SUMMARY:

- **View:** cat file.txt
- **Copy:** cp <source> <destination>
- **Move/Rename:** mv <source> <destination>
- **Remove:** rm file or rm -r dir
- **Install:** sudo apt update && sudo apt install pkg
- **Compile C:** gcc file.c -o output - ./output
- **Permissions:** chmod +x script.sh